



JDBC常用接口介绍

华为云 + 智能, 见未来

www.huaweicloud.com



前言

- JDBC是Java数据库连接(Java Database Connectivity)的简称，它是用于Java编程语言和数据库之间的标准Java API，本章将介绍GaussDB(for openGauss) 支持的JDBC驱动和常用接口。



课程目标

- 学完本课程后，您将能够：
 - 掌握JDBC的工作原理与主要功能；
 - 掌握常用的JDBC接口。



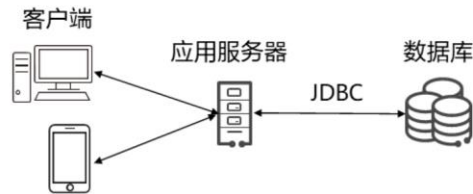
目录

1. JDBC概述

2. 常用接口介绍

JDBC概述

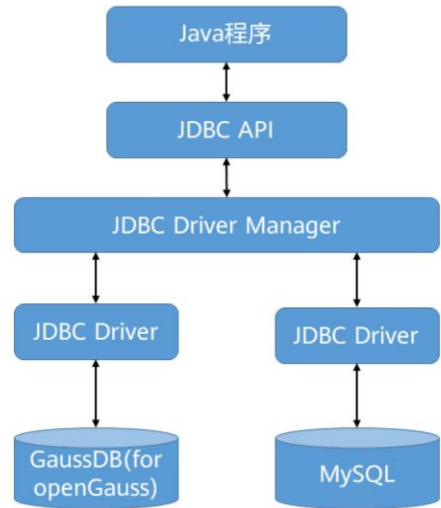
- JDBC是Java连接数据库（Java Database Connect）的简称，它用于Java语言的程序与数据库之间，提供连接各种常用数据库的能力。



- JDBC包含两个包：java.sql和javax.sql
 - java.sql：基本功能。这个包中的类和接口主要针对基本的数据库编程服务，如生成连接、执行语句以及准备语句和运行批处理查询等。同时也有一些高级的处理，比如批处理更新、事务隔离和可滚动结果集等。
 - javax.sql：扩展功能。它主要为数据库方面的高级操作提供了接口和类。如为连接管理、分布式事务和旧有的连接提供了更好的抽象，它引入了容器管理的连接池、分布式事务和行集（RowSet）等。

JDBC程序工作原理

- JDBC体系结构由三层组成：
 - JDBC API：提供应用程序到JDBC管理器连接，提供程序调用的接口与类。
 - JDBC Driver Manager：对JDBC各类驱动进行管理载入。
 - JDBC Driver：负责连接不同的数据库。



- 因此主要关注JDBC API的内容，至于JDBC Driver Manager和JDBC Driver都已集成进JDBC中，不需要关注。
- JDBC Driver由各个数据库厂商提供，不需要关注。

JDBC API

- JDBC API主要功能为：

- 同数据库建立连接；

- DriverManager：使用通信子协议将来自java应用程序的连接请求与适当的数据库驱动程序进行匹配，根据不同的数据库，管理JDBC驱动

- Connection：进行数据库连接，进行数据传输

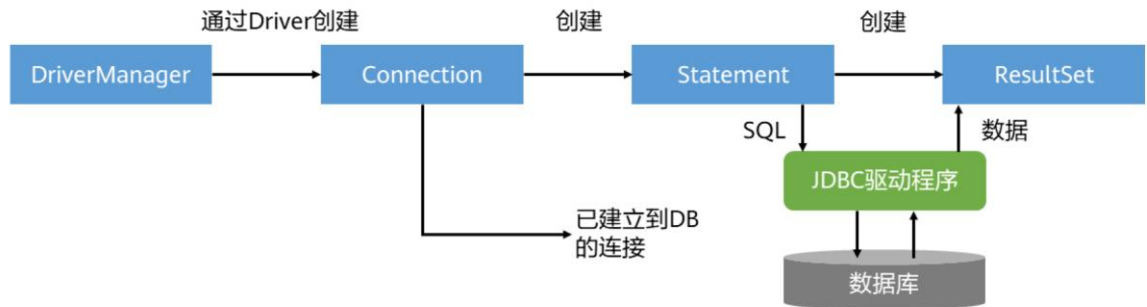
- 执行SQL语句；

- Statement：执行SQL语句。

- 处理结果；

- ResultSet：在使用Statement对象执行SQL查询后，这些对象保存从数据库检索的数据。

Java基于JDBC程序执行过程



大致步骤为：

- 1、注册JDBC驱动程序；2、建立到DB连接；3、创建SQL语句；4、执行SQL语句；5、处理结果；6、与数据库断开连接。

目录

1. JDBC概述

2. 常用接口介绍

Driver接口

- 用于管理JDBC驱动的服务类，程序中使用该类的主要功能是获取Connection对象。
- 主要方法有：

方法名	返回值类型	描述说明
acceptsURL(String url)	Boolean	查询驱动程序是否认为它可以打开到给定 URL 的连接
connect(String url, Properties info)	Connection	试图创建一个到给定 URL 的数据库连接
getMajorVersion()	int	获取此驱动程序的主版本号
getMinorVersion()	int	获得此驱动程序的次版本号
getPropertyInfo(String url, Properties info)	DriverPropertyInfo[]	获得此驱动程序的可能属性信息
jdbcCompliant()	Boolean	报告此驱动程序是否是一个真正的 JDBC Compliant驱动程序

Driver接口 - Connect方法

- 试图创建一个到给定 URL 的数据库连接。如果驱动程序识别出它是连接到给定 URL 的错误类型的驱动程序，那么它应该返回 "null"，因为在要求 JDBC 驱动程序管理器连接到给定 URL 时，它会将该 URL 依次传递给每个已加载的驱动程序。
- 如果要连接到给定 URL 的驱动程序是正确的驱动程序，但却无法连接到数据库，则该驱动程序应该抛出 `SQLException`。
- 可以使用 `java.util.Properties` 参数将任意字符串标记/值对作为连接参数传递。通常 `Properties` 对象中至少应该包括 "user" 和 "password" 属性。

```
Connection connect(String url, Properties info)  
    throws SQLException
```

- 参数：
 - url - 要连接到的数据库的 URL
 - info - 做为连接参数的任意字符串标记/值对的列表，通常至少应该包括 "user" 和 "password" 属性。
- 返回：表示到 URL 的连接的 `Connection` 对象
- 抛出：`SQLException` - 如果发生数据库访问错误

- URL的内容，可以参考https://support.huaweicloud.com/devg-opengauss/opengauss_devg_0110.html

DriverManager类

- DriverManager 类是用来管理数据库驱动的，在java.sql 包中大多数都是接口，DriverManager是为数不多的类之一。
- DriverManager是非常常用的一个类，最主要的功能就是获得数据库的连接，它定义了三个方法，用于创建数据库连接。差别在于参数的数量上。
 - DriverManager.getConnection(String url);
 - DriverManager.getConnection(String url, Properties info);
 - DriverManager.getConnection(String url, String user, String password);
- 最后一个带三个参数的 getConnection()方法是最常用的。
- 使用gsjdbc4.jar时，数据库URL（Uniform Resource Locator，统一资源定位符）连接描述符如下：
 - jdbc:postgresql://host:port/database
- 使用gsjdbc200.jar时，数据库URL连接描述符如下：
 - jdbc:gaussdb://host:port/database

- DriverManager.getConnection调用以driver.connect()获取数据库连接。但是不建议使用driver.connect()直接去进行连接，而应该使用DriverManager类进行连接。

Connection接口

- Connection接口表示应用程序与数据库的连接对象，在连接上下文中执行 SQL 语句并返回结果。每个Connection代表一个物理连接会话，想要访问数据库，必须先获得数据库连接。通过 Connection 对象，可以获得操作数据库的 Statement、PreparedStatement，CallableStatement 等对象。常用的方法如下：

方法名	返回值类型	描述说明
createStatement()	Statement	创建Statement对象
prepareStatement()	PreparedStatement	创建预处理对象preparedStatement
prepareCall(String sql)	CallableStatement	创建一个CallableStatement对象来调用数据库存储过程
commit()	void	使所有上一次提交/回滚后进行的更改成为持久更改，并释放此Connection对象当前持有的所有数据库锁
rollback()	void	取消在当前事务中进行的所有更改，并释放此Connection对象当前持有的所有数据库锁
close()	void	立即释放此Connection对象的数据库和JDBC资源，而不是等待它们被自动释放

- 同时使用Connection接口可以设置

Connection接口 - createStatement方法

- 创建一个 Statement 对象来将 SQL 语句发送到数据库。不带参数的 SQL 语句通常使用 Statement 对象执行。如果多次执行相同的 SQL 语句，使用 PreparedStatement 对象可能更有效。
- 使用返回的 Statement 对象创建的结果集在默认情况下类型为 TYPE_FORWARD_ONLY，并带有 CONCUR_READ_ONLY 并发级别。

```
Statement createStatement()  
    throws SQLException
```

- 返回：一个新的默认 Statement 对象
- 抛出：SQLException - 如果发生数据库访问错误，或者在关闭的连接上调用此方法
- 例如调用Connection的createStatement方法创建语句对象：

```
Connection conn = DriverManager.getConnection("url","user","password");  
Statement stmt = conn.createStatement();
```

Connection接口 - preparedStatement方法

- 创建一个 PreparedStatement 对象来将参数化的 SQL 语句发送到数据库。带有 IN 参数或不带有 IN 参数的 SQL 语句都可以被预编译并存储在 PreparedStatement 对象中，预编译语句是只编译和优化一次，可以有效地使用此对象来多次执行该语句。由于已经预先编译好，后续使用会减少执行时间。因此，如果多次执行一条语句，选择使用预编译语句。

```
PreparedStatement prepareStatement(String sql)  
    throws SQLException
```

- 参数: sql - 可能包含一个或多个 '?' IN 参数占位符的 SQL 语句
- 返回: 包含预编译 SQL 语句的新的默认 PreparedStatement 对象
- 抛出: SQLException - 如果发生数据库访问错误，或者在关闭的连接上调用此方法
- 例如调用Connection的prepareStatement方法创建预编译语句对象:

```
PreparedStatement pstmt = con.prepareStatement("UPDATE customer_t1 SET c_customer_name = ? WHERE  
c_customer_sk = 1");
```

Connection接口 - prepareCall方法

- 创建一个CallableStatement对象来调用数据库存储过程。CallableStatement对象提供了设置其IN和OUT参数的方法，以及用来执行调用存储过程的方法。

```
CallableStatement prepareCall(String sql)  
throws SQLException
```

- 参数: sql - 可以包含一个或多个 '?' 参数占位符的 SQL 语句。通常此语句是使用 JDBC 调用转义语法指定的
- 返回: 包含预编译 SQL 语句的新的默认 CallableStatement 对象
- 抛出: SQLException - 如果发生数据库访问错误，或者在关闭的连接上调用此方法
- 例如调用Connection的prepareCall方法创建调用语句对象:

```
Connection myConn = DriverManager.getConnection("url","user","password");  
CallableStatement cstmt = myConn.prepareCall("{? = CALL TESTPROC(?,?,?)}");
```


Connection接口 – 其他方法

- commit方法

- 使所有上一次提交/回滚后进行的更改成为持久更改，并释放此Connection对象当前持有的所有数据库锁。此方法只应该在已禁用自动提交模式时使用，与setAutoCommit方法使用相关。

- 例如：`connect.commit ();`

- rollback方法

- 取消在当前事务中进行的所有更改，并释放此 Connection 对象当前持有的所有数据库锁。此方法只应该在已禁用自动提交模式时使用，与setAutoCommit方法使用相关。

- 例如：`connect.rollback();`

- close方法

- 立即释放此 Connection 对象的数据库和 JDBC 资源，而不是等待它们被自动释放。在已经关闭的 Connection 对象上调用 close 方法无操作。

- 例如：`connect.close ();`

Statement接口

- 用于执行静态 SQL 语句并返回它所生成结果的对象。在默认情况下，同一时间每个 Statement 对象只能打开一个 ResultSet 对象。因此，如果读取一个 ResultSet 对象与读取另一个交叉，则这两个对象必须是由不同的 Statement 对象生成的。如果存在某个语句的打开的当前 ResultSet 对象，则 Statement 接口中的所有执行方法都会隐式关闭它。
- Statement 接口是用于执行SQL语句的工具接口，该对象既可用于执行DDL、DCL语句，也可用于执行DML语句，还可以用于执行SQL查询，返回查询到的结果集。接口常用方法如下：

方法名	返回值类型	描述说明
execute(String sql)	boolean	执行给定的 SQL 语句，该语句可能返回多个结果
executeQuery(String sql)	ResultSet	执行给定的 SQL 语句，该语句返回单个 ResultSet 对象
executeUpdate(String sql)	int	执行给定SQL语句，该语句可能为INSERT、UPDATE或DELETE语句，或者不返回任何内容的SQL语句（如DDL）。
close()	void	立即释放此 Statement 对象的数据库和 JDBC 资源，而不是等待该对象自动关闭时发生此操作

Statement接口 – execute方法

- 执行给定的 SQL 语句，该语句可能返回多个结果，execute 方法执行 SQL 语句并指示第一个结果的形式。然后，必须使用 `getResultSet` 或 `getUpdateCount` 来获取结果，使用 `getMoreResults` 来移动后续结果。

```
boolean execute(String sql)  
    throws SQLException
```

- 参数: sql - 任何 SQL 语句
- 返回: 如果第一个结果为 `ResultSet` 对象，则返回 `true`；如果其为更新计数或者不存在任何结果，则返回 `false`
- 抛出: `SQLException` - 如果发生数据库访问错误，或者在已关闭的 `Statement` 上调用此方法
- 例如使用execute方法创建test表:

```
String testTableSQL = "create table test (id int,name varchar(256));";  
Statement st = conn.createStatement();  
st.execute( testTableSQL );
```

Statement接口 – executeQuery方法

- 执行给定的 SQL 语句，该语句返回单个 ResultSet 对象，该方法只能用于执行查询语句。

```
ResultSet executeQuery(String sql)  
    throws SQLException
```

- 参数：sql - 要发送给数据库的 SQL 语句，通常为静态 SQL SELECT 语句
- 返回：包含给定查询所生成数据的 ResultSet 对象；永远不能为 null
- 抛出：SQLException - 如果发生数据库访问错误，在已关闭的 Statement 上调用此方法，或者给定 SQL 语句生成单个 ResultSet 对象之外的任何其他内容
- 例如，使用executeQuery查询test表中的数据：

```
ResultSet rs =stmt.executeQuery("select * from test");
```

Statement接口 – executeUpdate方法

- 执行给定SQL语句，该方法用于执行DML语句，并返回受影响的行数，同时也可以用于执行DDL语句，执行后返回0。

```
int executeUpdate(String sql)  
    throws SQLException
```

- 参数：sql - SQL数据操作语言（Data Manipulation Language，DML）语句，如INSERT、UPDATE或DELETE；或者不返回任何内容的SQL语句，如DDL语句。
- 返回：对于DML语句，返回行数；对于无返回值的SQL语句，返回0
- 抛出：SQLException - 如果在已关闭的Statement上调用此方法，或者给定的SQL语句生成ResultSet对象，则发生数据库访问错误
- 例如，使用executeUpdate方法更新test表中的数据：

```
int rc = stmt.executeUpdate("UPDATE TABLE TEST SET NAME='JERRY' WHERE ID = 1;");
```

PreparedStatement接口

- PreparedStatement接口表示预编译的 SQL 语句的对象，是Statement的子接口，它允许数据库预编译SQL语句（这些SQL语句通常都带有参数），以后每次只改变SQL命令的参数，避免数据库每次都需要编译SQL语句，因此性能更好。相对于Statement而言，使用PreparedStatement执行SQL语句时，无需再传入SQL语句，只要为预编译的SQL语句传入参数值即可。它比Statement主要多了如下方法：

方法名	返回值类型	描述说明
setXxx(int parameterIndex,Xxx value) (Xxx为数据类型)	void	该方法根据参数值的类型不同，选用不同的方法。传入的值根据索引传给SQL语句中指定位置的参数。如int型，则使用setInt方法；如date型，则使用setDate方法。
clearParameters()	void	立即清除当前参数值

- 此处列举的方法为主要的，方法与Statement对比后完全多出的方法。
- statement发送完整的Sql语句到数据库不是直接执行而是由数据库先编译，再运行。而PreparedStatement是先发送带参数的Sql语句，再发送一组参数值。如果是同构的sql语句，PreparedStatement的效率要比statement高。而对于异构的sql则两者效率差不多。
 - 同构：两个Sql语句可编译部分是相同的，只有参数值不同。
 - 异构：整个sql语句的格式是不同的
- 注意点：
 - 1、使用预编译的Statement编译多条Sql语句一次执行
 - 2、可以跨数据库使用，编写通用程序
 - 3、能用预编译时尽量用预编译

PreparedStatement接口示例

- 调用Connection的prepareStatement方法创建预编译语句对象

```
pstmt = con.prepareStatement("UPDATE test SET name = ? WHERE id = 1");
```

- 调用PreparedStatement的setShort设置参数

```
pstmt.setShort(1, (short)2);
```

- 调用PreparedStatement的executeUpdate方法执行预编译SQL语句

```
int rowcount = pstmt.executeUpdate();
```

- 调用PreparedStatement的close方法关闭预编译语句对象

```
pstmt.close();
```

CallableStatement接口

- CallableStatement接口用于执行SQL存储过程的接口，它是PreparedStatement的子接口。
- CallableStatement对象是通过调用Connection对象的prepareCall()方法创建的。prepareCall()方法有三种形式，既支持创建使用默认类型的ResultSet的CallableStatement对象，也支持开发人员创建指定类型的ResultSet的CallableStatement对象。这与创建Statement对象过程相似。
- 存储过程的运行是在数据库内部进行的，存储过程除了能查询并返回数据之外，存储过程还支持IN、OUT参数。所以，这也使得CallableStatement接口的使用方法与PreparedStatement稍有不同。

CallableStatement接口主要有以下几种：

方法名	返回值类型	描述说明
getXxx(int parameterIndex,Xxx value) (Xxx为数据类型)	Xxx	以Java编程语言中Xxx类型获取JDBC中Xxx类型的值。如int型，则使用getInt方法；以Java编程语言中int值的形式获取指定的JDBC INTEGER参数的值。
registerOutParameter(int parameterIndex, int type)	void	按顺序位置parameterIndex将OUT参数注册为JDBC类型sqlType

- 以下方法是从java.sql.Statement继承而来：close，execute，executeQuery，executeUpdate，getConnection，getResultSet，getUpdateCount，isClosed，setMaxRows，setFetchSize。
- 以下方法是从java.sql.PreparedStatement继承而来：addBatch，clearParameters，execute，executeQuery，executeUpdate，getMetaData，setBigDecimal，setBoolean，setByte，setBytes，setDate，setDouble，setFloat，setInt，setLong，setNull，setObject，setString，setTime，setTimestamp。

CallableStatement接口示例

- 调用Connection的prepareCall方法创建调用语句对象

```
Connection myConn = DriverManager.getConnection("url","user","password");  
CallableStatement cstmt = myConn.prepareCall("{? = CALL TESTPROC(?,?)}");
```

- 调用CallableStatement的setInt方法设置参数

```
cstmt.setInt(2, 50);  
cstmt.setInt(1, 20);
```

- 调用CallableStatement的registerOutParameter方法注册输出参数

```
cstmt.registerOutParameter(4, Types.INTEGER);
```

- 调用CallableStatement的execute执行方法调用

```
cstmt.execute();
```

- 调用CallableStatement的getInt方法获取输出参数

```
int out = cstmt.getInt(4);
```

- 调用CallableStatement的close方法关闭调用语句

```
cstmt.close();
```

- 在数据库中已创建了如下存储过程，它带有out参数。
- create or replace procedure testproc
- (
- psv_in1 in integer,
- psv_in2 in integer,
- psv_inout in out integer
-)
- as
- begin
- psv_inout := psv_in1 + psv_in2 + psv_inout;
- end;
- /

ResultSet接口

- ResultSet接口为结果集对象，该对象包含访问查询结果的方法，通常通过执行查询数据库的语句生成。
- ResultSet接口类似于一个临时表，ResultSet实例具有指向当前数据行的指针，指针开始的位置在第一条记录的前面，通过next()方法可以将指针向下移动；ResultSet可以通过列索引或列名获得列数据。常用方法如下：

方法名	返回值类型	描述说明
getXxx() (Xxx为数据类型)	void	该方法根据参数值的类型不同，选用不同的方法。获取此ResultSet对象的当前行的指定列值。如int型，则使用getInt方法；如date型，则使用getDate方法。
absolute(int row)	boolean	将结果集的记录指针移动到第row行，如果row是负数，则移动到倒数第row。如果移动后的记录指针指定一条有效记录，则该方法返回true。
first()	boolean	将ResultSet的记录指针定位到首行，如果移动后的记录指针指向一条有效记录，则方法返回true。
last()	boolean	将ResultSet的记录指针定位到最后一行，如果移动后的记录指针指向一条有效记录，则方法返回true。
previous()	boolean	将ResultSet的记录指针定位到上一行，如果移动后的记录指针指向一条有效记录，则方法返回true。
next()	boolean	将ResultSet的记录指针定位到下一行，如果移动后的记录指针指向一条有效记录，则方法返回true。
close()	void	释放ResultSet对象

ResultSet 接口用来表示查询结果集。当调用 Statement 的 executeQuery()方法时，就会得到一个 ResultSet 的对象。ResultSet 对象中包含根据查询语句查询出来的一个结果集，但是，实际上这些内容还是在数据库当中，还并没有真正的取出到虚拟机的内存中。ResultSet 其实是保存了一个指向其当前数据行的游标，需要使用 ResultSet 的方法让游标一行一行的向下移动，然后获取每一行的数据，所以在操作 ResultSet 对象期间，数据库连接不能关闭。

ResultSet接口参数类型

- resultSetType: 表示结果集的类型，具体有三种类型。
 - ResultSet.TYPE_FORWARD_ONLY: ResultSet结果集游标只能向前移动。是缺省值。
 - ResultSet.TYPE_SCROLL_SENSITIVE: ResultSet结果集的游标可以上下移动，当结果集变化时，当前结果集同步改变。
 - ResultSet.TYPE_SCROLL_INSENSITIVE: ResultSet结果集的游标可以上下移动，当结果集变化时，当前结果集不变。
- resultSetConcurrency: 表示结果集的并发，具体有两种类型。
 - ResultSet.CONCUR_READ_ONLY: 如果不从结果集中的数据建立一个新的更新语句，不能对结果集中的数据进行更新。
 - ResultSet.CONCUR_UPDATEABLE: 可改变的结果集。对于可滚动的结果集，可对结果集进行适当的改变。

ResultSetMetaData接口

- ResultSetMetaData接口用于收集ResultSet的所有元数据信息，例如列的类型和属性，列数，列的名称，列的数据类型等。
- ResultSetMetaData接口封装了描述ResultSet对象的数据，内部提供了大量的方法来获取ResultSet的信息。

方法名	返回值类型	描述说明
getColumnCount()	int	返回ResultSet对象中的列总数
getColumnName(int index)	String	返回指定列索引的列名
getColumnTypeName(int index)	String	返回指定索引的列类型名称
getTableName(int index)	String	返回指定列索引的表名

综合示例

```
public class SQLRetry {
    //创建数据库连接。
    public static Connection GetConnection(String username, String
passwd) {
    String driver = "org.postgresql.Driver";
    String sourceURL = "jdbc:postgresql://192.168.1.127:8000/postgres";
    Connection conn = null;
    try {
        //加载数据库驱动。
        Class.forName(driver).newInstance();
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }

    try {
        //创建数据库连接。
        conn = DriverManager.getConnection(sourceURL, username,
passwd);
        System.out.println("Connection succeed!");
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }

    return conn;
}
```

www.huaweicloud.com

```
//执行普通SQL语句，创建jdbc_test1表。
public static void CreateTable(Connection conn) {
    Statement stmt = null;
    try {
        stmt = conn.createStatement();
        Runtime.getRuntime().addShutdownHook(new ExitHandler(stmt));
    } catch (SQLException e) {
        e.printStackTrace();
    }
    //执行普通SQL语句。
    int rc2 = stmt
        .executeUpdate("DROP TABLE if exists jdbc_test1;");
    int rc1 = stmt
        .executeUpdate("CREATE TABLE jdbc_test1(col1 INTEGER, col2
VARCHAR(10));");
    stmt.close();
    } catch (SQLException e) {
        if (stmt != null) {
            try {
                stmt.close();
            } catch (SQLException e1) {
                e1.printStackTrace();
            }
        }
        e.printStackTrace();
    }
}
```



总结

- 本章通过对JDBC驱动的介绍，对其工作原理、使用过程及常用接口的使用有了基础的认识和熟悉。基于常规接口的组合可以应对数据的基础操控，完成业务逻辑的处理及封装。

思考题

1. （判断题） ResultSet.TYPE_SCROLL_SENSITIVE参数表示ResultSet结果集的游标可以上下移动，当结果集变化时，当前结果集不变。（ ）
 - A. True
 - B. False
2. （单选题）关于接口，以哪项描述正确的？（ ）
 - A. CallableStatement是PreparedStatement的父接口
 - B. PreparedStatement是CallableStatement的父接口
 - C. CallableStatement是Statement的父接口
 - D. PreparedStatement是Statement的父接口

- B。ResultSet.TYPE_SCROLL_SENSITIVE： ResultSet结果集的游标可以上下移动，当结果集变化时，当前结果集同步改变。
- B。PreparedStatement是CallableStatement的父接口

缩略语

- JDBC, Java Database Connectivity, Java数据库连接。
- API, Application Programming Interface, 应用程序接口。
- URL, Uniform Resource Locator, 统一资源定位系统。



感谢

版权所有©2022，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编，也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为云 + 智能，见未来

www.huaweicloud.com